Self-Attentive Sequential Recommendation

Wang-Cheng Kang, Julian McAuley
UC San Diego
{wckang,jmcauley}@ucsd.edu

arXiv:1808.09781v1 [cs.IR] 20 Aug 2018

***Abstract*—Sequential dynamics are a key feature of many modern recommender systems, which seek to capture the 'context' of users' activities on the basis of actions they have performed recently. To capture such patterns, two approaches have proliferated: Markov Chains (MCs) and Recurrent Neural Networks (RNNs). Markov Chains assume that a user's next action can be predicted on the basis of just their last (or last few) actions, while RNNs in principle allow for longer-term semantics to be uncovered. Generally speaking, MC-based methods perform best in extremely sparse datasets, where model parsimony is critical, while RNNs perform better in denser datasets where higher model complexity is affordable. The goal of our work is to balance these two goals, by proposing a self-attention based sequential model (SASRec) that allows us to capture long-term semantics (like an RNN), but, using an attention mechanism, makes its predictions based on relatively few actions (like an MC). At each time step, SASRec seeks to identify which items are 'relevant' from a user's action history, and use them to predict the next item. Extensive empirical studies show that our method outperforms various state-of-the-art sequential models (including MC/CNN/RNN-based approaches) on both sparse and dense datasets. Moreover, the model is an order of magnitude more efficient than comparable CNN/RNN-based models. Visualizations on attention weights also show how our model adaptively handles datasets with various density, and uncovers meaningful patterns in activity sequences.**

I. Introduction

The goal of sequential recommender systems is to combine personalized models of user behavior (based on historical activities) with some notion of 'context' on the basis of users' recent actions. Capturing useful patterns from sequential dynamics is challenging, primarily because the dimension of the input space grows exponentially with the number of past actions used as context. Research in sequential recommendation is therefore largely concerned with how to capture these high-order dynamics succinctly.

Markov Chains (MCs) are a classic example, which assume that the next action is conditioned on only the previous action (or previous few), and have been successfully adopted to characterize short-range item transitions for recommendation [1]. Another line of work uses Recurrent Neural Networks (RNNs) to summarize all previous actions via a hidden state, which is used to predict the next action [2].

Both approaches, while strong in specific cases, are somewhat limited to certain types of data. MC-based methods, by making strong simplifying assumptions, perform well in high-sparsity settings, but may fail to capture the intricate dynamics of more complex scenarios. Conversely RNNs, while expressive, require large amounts of data (an in particular *dense* data) before they can outperform simpler baselines.

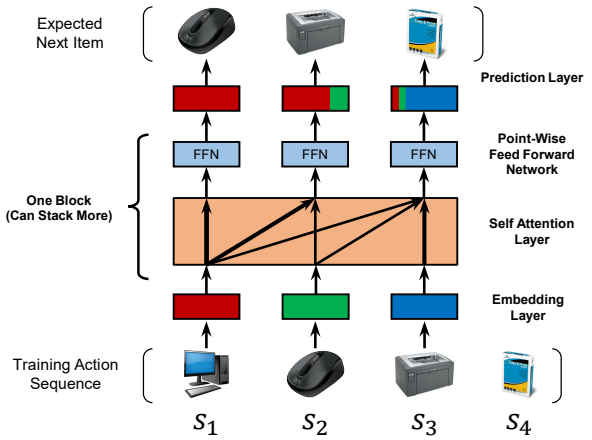

Figure 1: A simplified diagram showing the training process of SASRec. At each time step, the model considers all previous items, and uses attention to 'focus on' items relevant to the next action.

Recently, a new sequential model *Transfomer* achieved state-of-the-art performance and efficiency for machine translation tasks [3]. Unlike existing sequential models that use convolutional or recurrent modules, Transformer is purely based on a proposed attention mechanism called 'self-attention,' which is highly efficient and capable of uncovering syntactic and semantic patterns between words in a sentence.

Inspired by this method, we seek to apply self-attention mechanisms to sequential recommendation problems. Our hope is that this idea can address both of the problems outlined above, being on the one hand able to draw context from all actions in the past (like RNNs) but on the other hand being able to frame predictions in terms of just a small number of actions (like MCs). Specifically, we build a Self-Attention based Sequential Recommendation model (*SASRec*), which adaptively assigns weights to previous items at each time step (Figure 1).

The proposed model significantly outperforms state-of-the-art MC/CNN/RNN-based sequential recommendation methods on several benchmark datasets. In particular, we examine performance as a function of dataset sparsity, where model performance aligns closely with the patterns described above. Due to the self-attention mechanism, SASRec tends to consider long-range dependencies on dense datasets, while focusing on more recent activities on sparse datasets. This proves crucial for adaptively handling datasets with varying density.

Furthermore, the core component (i.e., the self-attention

block) of SASRec is suitable for parallel acceleration, resulting in a model that is an order of magnitude faster than CNN/RNN-based alternatives. In addition, we analyze the complexity and scalability of SASRec, conduct a comprehensive ablation study to show the effect of key components, and visualize the attention weights to qualitatively reveal the model's behavior.

II. Related Work

Several lines of work are closely related to ours. We first discuss general, followed by temporal, recommendation, before discussing sequential recommendation (in particular MCs and RNNs). Last we introduce the attention mechanism, especially the self-attention module which is at the core of our model.

A. General Recommendation

Recommender systems focus on modeling the compatibility between users and items, based on historical feedback (e.g. clicks, purchases, likes). User feedback can be *explicit* (e.g. ratings) or *implicit* (e.g. clicks, purchases, comments) [4], [5]. Modeling implicit feedback can be challenging due to the ambiguity of interpreting 'non-observed' (e.g. non-purchased) data. To address the problem, *point-wise* [4] and *pairwise* [5] methods are proposed to solve such challenges.

Matrix Factorization (MF) methods seek to uncover latent dimensions to represent users' preferences and items' properties, and estimate interactions through the inner product between the user and item embeddings [6], [7]. In addition, another line of work is based on Item Similarity Models (ISM) and doesn't explicitly model each user with latent factors (e.g. FISM [8]). They learn an item-to-item similarity matrix, and estimate a user's preference toward an item via measuring its similarities with items that the user has interacted with before.

Recently, due to their success in related problems, various deep learning techniques have been introduced for recommendation [9]. One line of work seeks to use neural networks to extract item features (e.g. images [10], [11], text [12], [13], etc.) for content-aware recommendation. Another line of work seeks to replace conventional MF. For example, NeuMF [14] estimates user preferences via Multi-Layer Perceptions (MLP), and AutoRec [15] predicts ratings using autoencoders.

B. Temporal Recommendation

Dating back to the *Netflix* Prize, temporal recommendation has shown strong performance on various tasks by explicitly modeling the timestamp of users' activities. TimeSVD++ [16] achieved strong results by splitting time into several segments and modeling users and items separately in each. Such models are essential to understand datasets that exhibit significant (short- or long-term) temporal 'drift' (e.g. 'how have movie preferences changed in the last 10 years,' or 'what kind of businesses do users visit at 4pm?', etc.) [16]–[18]. Sequential recommendation (or next-item recommendation) differs slightly from this setting, as it only considers the order of actions, and models sequential patterns which are independent of time. Essentially, sequential models try to model the 'context' of users' actions based on their recent activities, rather than considering temporal patterns *per se*.

C. Sequential Recommendation

Many sequential recommender systems seek to model item-item transition matrices as a means of capturing sequential patterns among successive items. For instance, FPMC fuses an MF term and an item-item transition term to capture long-term preferences and short-term transitions respectively [1]. Essentially, the captured transition is a first-order Markov Chain (MC), whereas higher-order MCs assume the next action is related to several previous actions. Since the last visited item is often the key factor affecting the user's next action (essentially providing 'context'), first-order MC based methods show strong performance, especially on sparse datasets [19]. There are also methods adopting high-order MCs that consider more previous items [20], [21]. In particular, Convolutional Sequence Embedding (Caser), a CNN-based method, views the embedding matrix of L previous items as an 'image' and applies convolutional operations to extract transitions [22].

Other than MC-based methods, another line of work adopts RNNs to model user sequences [2], [23]–[25]. For example, GRU4Rec uses Gated Recurrent Units (GRU) to model click sequences for session-based recommendation [2], and an improved version further boosts its Top-N recommendation performance [26]. In each time step, RNNs take the state from the last step and current action as its input. These dependencies make RNNs less efficient, though techniques like 'session-parallelism' have been proposed to improve efficiency [2].

D. Attention Mechanisms

Attention mechanisms have been shown to be effective in various tasks such as image captioning [27] and machine translation [28], among others. Essentially the idea behind such mechanisms is that sequential outputs (for example) each depend on 'relevant' parts of some input that the model should focus on successively. An additional benefit is that attention-based methods are often more interpretable. Recently, attention mechanisms have been incorporated into recommender systems [29]–[31]. For example, Attentional Factorization Machines (AFM) [30] learn the importance of each feature interaction for content-aware recommendation.

However, the attention technique used in the above is essentially an *additional* component to the original model (e.g. attention+RNNs, attention+FMs, etc.). Recently, a purely attention-based sequence-to-sequence method, Transfomer [3], achieved state-of-the-art performance and efficiency on machine translation tasks which had previously been dominated by RNN/CNN-based approaches [32], [33]. The Transformer model relies heavily on the proposed 'self-attention' modules to capture complex structures in sentences, and to retrieve relevant words (in the source language) for generating the next word (in the target language). Inspired by Transformer, we seek to build a new sequential recommendation model based upon the self-attention approach, though the problem of sequential recommendation is quite different from machine translation, and requires specially designed models.

Table I: Notation.

Notation	Description
$\mathcal{U}, \mathcal{I}$	user and item set
$\mathcal{S}^u$	historical interaction sequence for a user u: $(\mathcal{S}^u_1, \mathcal{S}^u_2, ..., \mathcal{S}^u_{\lvert\mathcal{S}^u\rvert})$
$d \in \mathbb{N}$	latent vector dimensionality
$n \in \mathbb{N}$	maximum sequence length
$b \in \mathbb{N}$	number of self-attention blocks
$\mathbf{M} \in \mathbb{R}^{\lvert\mathcal{I}\rvert \times d}$	item embedding matrix
$\mathbf{P} \in \mathbb{R}^{n \times d}$	positional embedding matrix
$\widehat{\mathbf{E}} \in \mathbb{R}^{n \times d}$	input embedding matrix
$\mathbf{S}^{(b)} \in \mathbb{R}^{n \times d}$	item embeddings after the b-th self-attention layer
$\mathbf{F}^{(b)} \in \mathbb{R}^{n \times d}$	item embeddings after the b-th feed-forward network

III. Methodology

In the setting of sequential recommendation, we are given a user's action sequence $\mathcal{S}^u = (\mathcal{S}^u_1, \mathcal{S}^u_2, \ldots, \mathcal{S}^u_{|\mathcal{S}^u|})$, and seek to predict the next item. During the training process, at time step t, the model predicts the next item depending on the previous t items. As shown in Figure 1, it will be convenient to think of the model's input as $(\mathcal{S}^u_1, \mathcal{S}^u_2, \ldots, \mathcal{S}^u_{|\mathcal{S}^u|-1})$ and its expected output as a 'shifted' version of the same sequence: $(\mathcal{S}^u_2, \mathcal{S}^u_3, \ldots, \mathcal{S}^u_{|\mathcal{S}^u|})$. In this section, we describe how we build a sequential recommendation model via an embedding layer, several self-attention blocks, and a prediction layer. We also analyze its complexity and further discuss how SASRec differs from related models. Our notation is summarized in Table I.

A. Embedding Layer

We transform the training sequence $(\mathcal{S}^u_1, \mathcal{S}^u_2, ..., \mathcal{S}^u_{|\mathcal{S}^u|-1})$ into a fixed-length sequence $s = (s_1, s_2, \ldots, s_n)$, where n represents the maximum length that our model can handle. If the sequence length is greater than n, we consider the most recent n actions. If the sequence length is less than n, we repeatedly add a 'padding' item to the left until the length is n. We create an item embedding matrix $\mathbf{M} \in \mathbb{R}^{|\mathcal{I}| \times d}$ where d is the latent dimensionality, and retrieve the input embedding matrix $\mathbf{E} \in \mathbb{R}^{n \times d}$, where $\mathbf{E}_i = \mathbf{M}_{s_i}$. A constant zero vector $\mathbf{0}$ is used as the embedding for the padding item.

Positional Embedding: As we will see in the next section, since the self-attention model doesn't include any recurrent or convolutional module, it is not aware of the positions of previous items. Hence we inject a learnable position embedding $\mathbf{P} \in \mathbb{R}^{n \times d}$ into the input embedding:

$$\widehat{\mathbf{E}} = \begin{bmatrix} \mathbf{M}_{s_1} + \mathbf{P}_1 \\ \mathbf{M}_{s_2} + \mathbf{P}_2 \\ \cdots \\ \mathbf{M}_{s_n} + \mathbf{P}_n \end{bmatrix} \tag{1}$$

We also tried the fixed position embedding as used in [3], but found that this led to worse performance in our case. We analyze the effect of the position embedding quantitatively and qualitatively in our experiments.

B. Self-Attention Block

The scaled dot-product attention [3] is defined as:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d}}\right)\mathbf{V}, \tag{2}$$

where $\mathbf{Q}$ represents the queries, $\mathbf{K}$ the keys and $\mathbf{V}$ the values (each row represents an item). Intuitively, the attention layer calculates a weighted sum of all values, where the weight between query i and value j relates to the interaction between query i and key j. The scale factor $\sqrt{d}$ is to avoid overly large values of the inner product, especially when the dimensionality is high.

Self-Attention layer: In NLP tasks such as machine translation, attention mechanisms are typically used with $\mathbf{K} = \mathbf{V}$ (e.g. using an RNN encoder-decoder for translation: the encoder's hidden states are keys and values, and the decoder's hidden states are queries) [28]. Recently, a self-attention method was proposed which uses the same objects as queries, keys, and values [3]. In our case, the self-attention operation takes the embedding $\widehat{\mathbf{E}}$ as input, converts it to three matrices through linear projections, and feeds them into an attention layer:

$$\mathbf{S} = \text{SA}(\widehat{\mathbf{E}}) = \text{Attention}(\widehat{\mathbf{E}}\mathbf{W}^Q, \widehat{\mathbf{E}}\mathbf{W}^K, \widehat{\mathbf{E}}\mathbf{W}^V), \tag{3}$$

where the projection matrices $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V \in \mathbb{R}^{d \times d}$. The projections make the model more flexible. For example, the model can learn asymmetric interactions (i.e., <query i, key j> and <query j, key i> can have different interactions).

Causality: Due to the nature of sequences, the model should consider only the first t items when predicting the $(t+1)$-st item. However, the t-th output of the self-attention layer ($\mathbf{S}_t$) contains embeddings of subsequent items, which makes the model ill-posed. Hence, we modify the attention by forbidding all links between $\mathbf{Q}_i$ and $\mathbf{K}_j$ $(j > i)$.

Point-Wise Feed-Forward Network: Though the self-attention is able to aggregate all previous items' embeddings with adaptive weights, ultimately it is still a linear model. To endow the model with nonlinearity and to consider interactions between different latent dimensions, we apply a point-wise two-layer feed-forward network to all $\mathbf{S}_i$ identically (sharing parameters):

$$\mathbf{F}_i = \text{FFN}(\mathbf{S}_i) = \text{ReLU}(\mathbf{S}_i\mathbf{W}^{(1)} + \mathbf{b}^{(1)})\mathbf{W}^{(2)} + \mathbf{b}^{(2)}, \tag{4}$$

where $\mathbf{W}^{(1)}, \mathbf{W}^{(2)}$ are $d \times d$ matrices and $\mathbf{b}^{(1)}, \mathbf{b}^{(2)}$ are d-dimensional vectors. Note that there is no interaction between $\mathbf{S}_i$ and $\mathbf{S}_j$ $(i \neq j)$, meaning that we still prevent information leaks (from back to front).

C. Stacking Self-Attention Blocks

After the first self-attention block, $\mathbf{F}_i$ essentially aggregates all previous items' embeddings (i.e., $\widehat{\mathbf{E}}_j, j \leq i$). However, it might be useful to learn more complex item transitions via another self-attention block based on $\mathbf{F}$. Specifically, we stack the self-attention block (i.e., a self-attention layer and a feed-forward network), and the b-th $(b > 1)$ block is defined as:

$$\begin{aligned} \mathbf{S}^{(b)} &= \text{SA}(\mathbf{F}^{(b-1)}), \\ \mathbf{F}^{(b)}_i &= \text{FFN}(\mathbf{S}^{(b)}_i), \quad \forall i \in \{1, 2, \ldots, n\}, \end{aligned} \tag{5}$$